

6. Programación funcional

Last edited time	October 14, 2026 8:55 PM
Practica	☐
Teoria	☑

Paradigma funcional

Paradigmas de programación

▼ Procedimental

- La solución se estructura como un programa lineal.
- El programa recibe una entrada y se le indica qué hacer con esta en cada paso.

▼ Vectorial

- Principal uso en programas matemáticos donde existe un paralelismo implícito en los cálculos.
- El compilador se encarga de realizar paralelismo en las partes donde es posible distribuir el trabajo.

▼ Declarativa

- Usuario declara problema a resolver → computador determina la mejor manera de resolver el programa de manera eficiente.
- ej. lenguajes de bases de datos, optimización, etc.

▼ Orientada a Objetos

- Programas modelan las funcionalidades a través de interacciones entre objetos.

▼ **Funcional**

- Corresponde a **programación procedimental de alto nivel.**
- Corresponde a un conjunto de funciones que **no tienen estado**, es decir, la salida depende solamente de los datos de entrada y no de otras variables externas.

Funciones puras

▼ **Pureza**

- Mide el **grado de dependencia** de la función hacia el sistema o el programa, es decir, el contexto en el cual se está ejecutando dicha función.
- + dependencia = - pureza
- + independencia = + pureza

- **Una función pura retorna lo mismo para el mismo input y no posee efectos secundarios.**

▼ Ejemplos de efectos secundarios

- Modificar las estructuras que recibió por referencia (ej. objetos, arreglos, etc.).
- Modificar variables globales o externas
- Operaciones de I/O (escribir en consola, leer o guardar archivos en sistema)

Consecuencia de la programación funcional

▼ No se modifican elementos externos a una función

- Si un valor o estructura cambia inesperadamente, solo tenemos que revisar las funciones impuras.

- El output depende solamente del input
  - Si el programa recuerda el input de una función pura, no necesita llamar nuevamente a la función.

    ▼ Esto se puede aprovechar usando el decorador `@cache` del módulo `functools`

```python
from functools import cache

@cache
def fibonacci_cached(n: int) -> int:
    print('buscando el número', n)
    if n < 2:
        return n
    return fibonacci_cached(n-1) + fibonacci_cached(n-2)

def fibonacci(n: int) -> int:
    print('buscando el número', n)
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
print("funcion normal:\n")
fibonacci(4)
print("\n\n funcion con cache:\n")
fibonacci_cached(4)
```

▼ Iteraciones como `for` o `while` se vuelven más amigables de modelar.

Funciones generadoras

- Las funciones en Python tienen la posibilidad de funcionar como **generadores** a través de la sentencia `yield`
  - `yield value` se encarga de retornar `value` y en la próxima llamada de la función, la ejecución de esta va a continuar desde esta línea.

▼ Basta con que una función tenga un `yield` para que sea considerada una **función generadora,** es decir, que retorna un generador.

- No es necesario que se alcance a ejecutar la línea con el `yield`
- Si una función generadora tiene `yield` y `return` , pero no alcanza el `yield` cuando se ejecuta, entonces la función va a retornar un generador vacío.
  - Si uno intenta recorrerlo le va a lanzar `StopIteration` junto con el valor que “retornó” el `return`
- Como un generador implementa `__iter__` , podemos ocupar un `for` o `next` para recorrer sus elementos.

📌 La anotación de tipo para un generador se hace con `Generator` del módulo `typing`

- Usos de funciones generadoras

  ▼ Manejar datos de gran tamaño, como **archivos**.

```python
from os import path
from typing import Generator


def cargar_archivo(path: str) -> Generator:
    """Lee el contenido de un archivo, sin cargar toda la información del mismo
```

```
en memoria."""

    with open(path, encoding='utf-8') as file:
        for line in file:
            yield line


lenguas_extintas = cargar_archivo(path.join('data', 'lenguas_extintas.csv'))

for i in range(5):
    print(next(lenguas_extintas))
```

▼ **Iterar por EDDs propias**, sin necesidad de crear una clase iteradora.

```
from typing import Generator


class IterableListaNumeros:
    def __init__(self, objeto: list) -> None:
        self.objeto = objeto.copy()

    def __iter__(self) -> Generator:
        while self.objeto:
            yield self.objeto.pop(0)
```

Este método `__iter__` retorna un generador, pero **ojo** que modifica la lista que contiene. [Si quisieras evitar modificarla, podrías ocupar `deepcopy` del módulo `copy`]

▼ Crear **generadores a partir de iterables**

- `yield from` permite crear un generador a partir de algún otro iterable, sin necesidad de ocupar algún tipo de bucle o loop para recorrer sus elementos.
- Como un generador es un iterable, también se puede hacer un generador a partir de otro generador.

▼ Podemos enviar objetos a la función generadora a través del método `send()`

- `variable = yield valor` permite que a `variable` se le asigne el argumento que pasamos con `send(value)`
- No es posible ocupar `send()` con un valor distinto de `None` en la primera iteración. En otras palabras, sólo se puede ocupar con libertad **después** de la primera iteración.
  - Si uno hace `next` , esto equivale a hacer `send(None)`

▼ Cuando uno ocupa el método, la función reanuda su ejecución desde esa línea (`variable = yield valor`), le asigna el valor que pasamos a `variable` y continua con el resto del código, hasta que encuentre otro `yield` y retorne el siguiente valor.

- Esto quiere decir que `generador.send(value)` retorna el siguiente valor de `generador`, contando la modificación que haya ocurrido después de pasarle `value`

Funciones que usan iterables

📌 El método `join` de Python funciona con cualquier objeto **iterable**, siempre y cuando todos los elementos devueltos por este sean **strings.**

`map (funcion, iterable1, ...)`

- `map` recibe una **función** y al menos un **iterable**.

    - La cantidad de iterables debe corresponder a la cantidad de parámetros que recibe la función.
    - Si los iterables son de distintos largos, `map` se va guiar por el de menor largo.
- Retorna un iterador que resulta de **aplicar la función sobre cada elemento del iterable** por separado.

`filter(funcion, iterable)`

- `filter` recibe una **función (**que retorne un **booleano)** y un **iterable**.
- Retorna un generador que entrega los elementos del iterable donde la función retorna `True`
    - Para filtrar por `False` se puede ocupar `itertools.filterfalse()`

`reduce(funcion, iterable)`

- `from` `functools` `import` `reduce`
- `reduce` recibe una **función** que toma dos valores, y un **iterable**.

  ▼ Es posible agregarle un **inicializador,** es decir, el primer elemento para que procese

    - `reduce(funcion, iterable, inicializador)`
- Retorna lo que resulta de aplicar la función `f` al iterable `[s1,s2,s3,...,sn]` de la forma `f(f(f(f(s1,s2),s3),s4),s5)...`

  ▼ Consiste en aplicar sucesivamente la función `f(x,y)` donde `x` es el resultado acumulado e `y` es un elemento de la secuencia.

```
def reduce(función, iterable, inicializador):
    acumulador = inicializador  # Toma el valor inicial ({})

    for elemento in iterable:    # Recorre el generador
        # El retorno de la función reescribe la variable 'acumulador'
        acumulador = función(acumulador, elemento)

    return acumulador
```

- Ejemplos de uso

  ▼ Aplanar listas

```
def concatenar_listas(x: list, y: list) -> list:
    return x + y


lista_con_listas = [[1, 2], [3, 4], [5, 6], [7, 8, 9]]
lista_aplanada = reduce(concatenar_listas, lista_con_listas)
lista_aplanada
```

  ▼ Intersección o unión de varios sets

```
def operación_unión(x: set, y: set) -> set:
    return x | y

def operación_intersección(x: set, y: set) -> set:
    return x & y


conjuntos = [{3, 5, 1}, {4, 3, 1}, {1, 2, 5}, {9, 5, 4, 1}]

resultado_unión = reduce(operación_unión, conjuntos)
resultado_intersección = reduce(operación_intersección, conjuntos)
```

```
print("Unión:", resultado_unión)
print("Intersección:", resultado_intersección)
```

▼ Duplicación de elementos

```
def duplicar_elementos(x: int, y: int) -> int:
    # Recuerda que el * se usa aquí para desempaquetar valores
    return *x, *(y for _ in range(y))

reduce(duplicar_elementos, lista, [])
```

▼ Cálculo de mínimos o máximos

- ⚠️ La forma más simple es ocupar las funciones `max()` y `min()` que reciben un iterable o varios argumentos independientes. Sin embargo, también se puede implementar con `reduce()`

```
# Hecho así sólo como ejemplo pedagógico
def máximo(x: int, y:int) -> int:
    if x >= y:
        return x
    return y


reduce(máximo, resultado_unión)
```

- Consideraciones de uso

  ▼ Cantidad de elementos que recibe `reduce`

  - Si el iterable está vacío y no agregamos un inicializador, entonces lanza `TypeError`
    - Si le pasamos un inicializador, entonces no lanza error.
  - Si solamente le pasamos el iterable y este tiene 1 elemento, entonces lo va a retornar intacto.

  ▼ Operaciones no-conmutativas

  - Dependiendo de la operación, podríamos necesitar fijar el orden del iterable para obtener el resultado que deseamos.
  - Especial consideración a **sets** u otros iterables no ordenados.

📌

- Un `set` crea una **tabla** fija en memoria.
- Para decidir en qué casilla guardar un elemento `x`, Python usa la fórmula `hash(x) % tamaño_tabla`.
- En el caso de **números enteros**, el hash es su propio valor. Por lo tanto, un set de puros números enteros, siempre se va a guardar en el **mismo orden**.
- En strings y otros tipos de elementos, el hash es variable entre cada ejecución.

Funciones `lambda` o anónimas

- Se escriben como `lambda <parámetros> : <retorno>`
- Puedes retornar lo que sea, siempre y cuando uses la notación correcta.
- Permiten escribir funciones simples sin necesidad de `def`
  → especialmente útil para funciones como `map` , `reduce` , `filter` , etc.

`enumerate`

- Recibe un **iterable**.
- Retorna un objeto de tipo `enumerate`, que se comporta de manera similar a un generador (permite `next` y `for`).
  - Sus elementos son tuplas de 2 objetos, donde el primero es el **índice** y el segundo el **ítem** original.

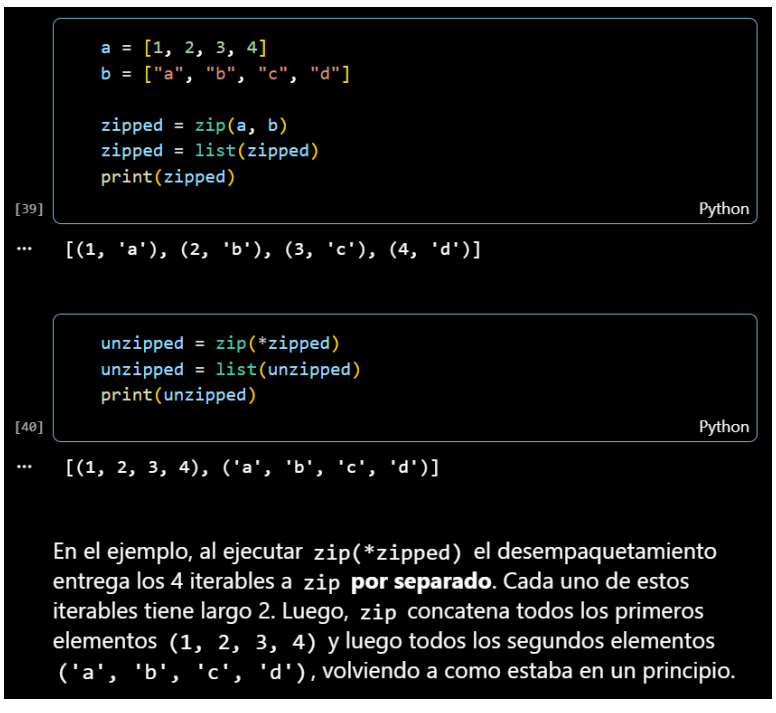
`zip`

- Toma dos o más secuencias o iterables y retorna un objeto de tipo `zip` que entrega **tuplas**, donde cada tupla está formada por los **elementos i-ésimos** de cada una de las secuencias o iterables.
  - El objeto que retorna se comporta de manera similar a un generador.
- La cantidad de elementos que retorna este iterador es igual al **menor de los largos** de las secuencias o iterables.

  ▼ `zip_longest()` permite que se trabaje hasta recorrer todo el iterable más largo

  - `from itertools import zip_longest` → `zip_longest(*iterables, fillvalue=None)`
  - En caso de haber un iterable de largo menor, los valores faltantes se reemplazan con lo que se especifique en `fillvalue`

▼ Es posible "**revertir**" lo que hace `zip` usando el operador `*`



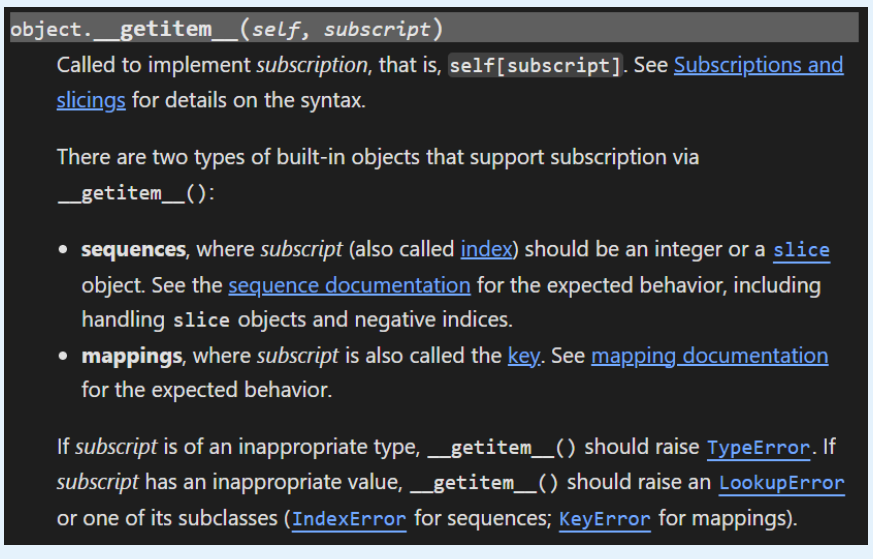
Aplicaciones

`reversed(objeto)`

- Retorna un **iterador invertido.**
- El objeto debe tener implementado el método `__reversed__()` o soportar el protocolo de **secuencias** (el método `__len__()` y el método `__getitem__()` que tome argumentos enteros desde `0`).



▼ **Cómo implementar `__getitem__()`**



`sum(), min(), max(), all(), any()`

- Todas estas funciones aceptan iterables como argumento.
- `all()` retorna `True` si todos los elementos del iterable son `True` . Así mismo, `any()` retorna `True` si existe algún elemento en el iterable que sea `True`
  - Se pueden combinar con una función que retorna un bool → `all(map(fx, iterable))`

▼ `sum()` se puede ocupar para calcular el largo de un generador

```
largo = sum(1 for _ in generador)
```

`itertools`

- Librería que contiene múltiples funciones que trabajan con iteradores
  - Funciones que entregan iteradores infinitos

▼ `count(start=0, step=1)`

- Hace un iterador que permite contar desde un número `start` en `step` pasos, de forma indefinida.

▼ `cycle(iterable)`

- Hace un iterador infinito de `iterable`
- Puede necesitar mucha memoria dependiendo del largo del iterable.

▼ `repeat(objeto, times=None)`

- Hace un iterador que retorna `objeto` de manera indefinida (si no se especifica `times`).

▼ Ejemplo de uso: pasarle valores constantes a `map` o `zip`

```
>>> list(map(pow, range(10), repeat(2)))
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

- Funciones orientadas en la combinatoria

▼ `product(*iterables, repeat=1)`

- Retorna el producto cartesiano entre los iterables que recibe de argumento.
- `product(A,B)` equivale a `((x,y) for x in A for y in B)`

▼ Para hacer el producto cartesiano con el mismo iterable se especifica con `repeat`

- `product(A, repeat=4)` equivale a `product(A, A, A, A)`

▼ `permutations(iterable, r=None)`

- Retorna permutaciones sucesivas de largo `r` desde el `iterable`
- Si no se especifica `r` (`r=None`), entonces se considera el largo del iterable.
- Los elementos son tratados como únicos en base a su **posición, no por su valor.**
  - Si los elementos del iterable son diferentes entre sí, entonces no va a haber elementos repetidos en una permutación.

▼ `combinations(iterable, r)`

- Retorna subsecuencias de elementos de largo `r` desde el `iterable`
- Los elementos son tratados como únicos en base a su **posición, no por su valor.**

        - Si los elementos del iterable son diferentes entre sí, entonces no va a haber elementos repetidos en una combinación.
    - La función `combinations_with_replacement(iterable, r)` funciona de manera similar, pero permite que los elementos se repitan más de una vez.

`collections`

- Librería que contiene múltiples EDDs para trabajar con colecciones de datos.
    - `namedtuple` , `deque` , `defaultdict` , `Counter` , etc.
- `Counter`
    - Subclase de **diccionario** que permite **contar los elementos de un iterable.**
    - Las llaves corresponden a los elementos del iterable y los valores a los recuentos de cada uno de estos.
        - Los elementos del iterable deben ser **hasheables.**
    - Se crea de la forma `Counter(iterable)`
    - Tiene los métodos de los diccionarios e incluso otros adicionales:
        - `contador.most_common(n)` retorna los `n` elementos más repetidos. Si no se especifica `n` , entonces devuelve todos los elementos.